



TECHNICAL DESIGN + DEVELOPER HANDBOOK

Persona 3 Reload AI Guide

A source-backed conversational companion for Persona 3 Reload

Technical Design Document | Software Requirements | Architecture Reference

Developer Handbook | Portfolio Case Study | Engineering Retrospective

DOCUMENT SCOPE

This document describes the implementation present in the local repository as of June 6, 2026. It distinguishes active production paths, retained prototypes, current limitations, and planned extensions. Secret values are intentionally excluded.

Prepared for the solo developer and future maintainers

Document Control

Field	Value
Project	Persona 3 Reload AI Guide / Tartarus Guide RAG
Repository	Nchaudry123/Tartarus-Guide
Document date	June 6, 2026
Audience	Developer, reviewer, portfolio reader, future contributor
Status	Current implementation reference; some working-tree improvements are not guaranteed to match the latest deployed build
Primary stack	Next.js, TypeScript, Supabase Postgres, pgvector, Hugging Face embeddings, Groq-compatible chat generation
Design preset	compact_reference_guide with named navy/cyan project palette override
Distribution	Local PDF only; no upload performed

IMPORTANT IMPLEMENTATION NOTE

The active Next.js page reads the shared static HTML shell and hydrates it with public JavaScript and CSS. A React component prototype remains in the repository as an alternate implementation path. This handbook documents both and labels the active path explicitly.

Contents

Open this document in Word and update fields to refresh the table of contents.

- 1. Project Overview
- 2. Development Goals and Requirements
- 3. Complete Feature Inventory
- 4. System Architecture
- 5. RAG System Documentation
- 6. Knowledge Base Documentation
- 7. User Experience Design
- 8. Database Documentation
- 9. API Documentation
- 10. AI System Design
- 11. Challenges and Engineering Decisions
- 12. Lessons Learned

- 13. Future Feature Roadmap
- 14. Scalability Considerations
- 15. Portfolio Case Study
- 16. Developer Retrospective
- Appendices: Runbook, Environment, File Map, Testing, Glossary

1. Project Overview

Why this project exists, who it serves, and what the experience is designed to feel like.

1.1 Project Purpose

Persona 3 Reload AI Guide is a personal passion project that turns conventional game-guide material into a conversational companion. A player can describe a problem in natural language, receive a direct strategy answer, and continue the conversation without translating the problem into exact guide keywords.

The project also functions as a learning vehicle for full-stack TypeScript development, production API design, data ingestion, vector search, structured fact extraction, prompt engineering, responsive interaction design, and responsible use of third-party guide sources.

1.2 Problem Being Solved

Traditional guides are accurate but fragmented. A player often needs to identify the right article, scan navigation-heavy pages, compare multiple sources, interpret terminology, and mentally combine details across bosses, schedules, Personas, requests, and Tartarus. This friction is especially visible on mobile devices during active play.

The guide reduces that cognitive cost. It accepts questions such as 'my team feels weak' or 'what should I do before the next full moon?', identifies the likely intent, asks for missing player context when needed, retrieves relevant evidence for exact claims, and translates that evidence into an actionable answer.

1.3 Target Users

- New players who need spoiler-aware explanations and do not yet know the game's terminology.
- Returning players who want quick weakness, fusion, schedule, request, or party advice.
- Completion-focused players who need dates, prerequisites, Social Link choices, rewards, and achievement planning.
- Mobile users who want a readable answer while playing rather than a long article search.
- The developer, who uses the project to learn modern AI application architecture through a concrete domain.

1.4 Core User Experience

The intended experience is closer to talking with an experienced Persona player than querying a search engine. The assistant should greet normally, remember recent context, recognize vague situations, ask

targeted follow-up questions, avoid spoilers, provide compact strategic guidance, and reserve citations for an optional source drawer.

User input	Expected assistant behavior
Hi	Respond conversationally without triggering retrieval.
My team feels weak	Ask level, active party, difficulty, or immediate blocker before recommending changes.
What is Dancing Hand weak to?	Prioritize a structured affinity fact; do not guess from a broadly related article.
What should I do today?	Use known month/date and priorities, or request the missing calendar context.
Help with Priestess	Retrieve boss mechanics and deliver a short party and turn-plan recommendation with spoiler restraint.

1.5 Personal Motivation

The project deliberately avoids a startup narrative. Its value is the craft: making a niche guide more enjoyable, understanding where RAG succeeds and fails, designing a distinctive interface, and producing a portfolio-quality system with enough technical depth to expose real engineering tradeoffs.

2. Development Goals and Requirements

The engineering and experience targets that shape implementation decisions.

2.1 Technical Goals

- Build an end-to-end RAG system rather than a thin wrapper around an LLM.
- Learn semantic embeddings, vector similarity search, hybrid retrieval, and structured fact modeling.
- Create a safe server-side boundary for privileged Supabase and model credentials.
- Develop a repeatable ingestion pipeline with discovery, robots checks, rate limiting, caching, cleaning, deduplication, embeddings, fact extraction, and audits.
- Separate exact game facts from broad guide prose so weaknesses, dates, floors, and recipes are not generated from vague similarity alone.
- Build a conversational control layer that can answer directly, clarify, or retrieve according to the user's intent.
- Support deployment as an integrated Next.js application while retaining a static preview mode for rapid UI iteration.

2.2 User Experience Goals

- Reduce time spent navigating and scanning traditional guide pages.
- Allow natural, incomplete, and conversational questions.
- Provide short, useful responses first, with details and sources available on demand.
- Keep the visual identity consistent with Persona 3 Reload without sacrificing mobile usability.
- Avoid revealing retrieval mechanics in normal answer prose.
- Mark uncertainty and ask one useful question instead of inventing unsupported facts.
- Preserve player progress context during the current browser session.

2.3 Functional Requirements

ID	Requirement	Current status
FR-01	Accept free-form chat questions	Implemented
FR-02	Classify game intent and ambiguity	Implemented in API controller
FR-03	Maintain recent conversation and player context	Implemented per browser session
FR-04	Retrieve exact structured facts and semantic chunks	Implemented; coverage still growing

ID	Requirement	Current status
FR-05	Generate a natural expert response using Groq	Implemented
FR-06	Expose citations without cluttering the answer	Implemented as collapsible source details
FR-07	Avoid unsupported exact weakness claims	Implemented guard; dependent on fact coverage
FR-08	Operate in static/mock mode without credentials	Implemented
FR-09	Support mobile viewport and keyboard behavior	Implemented and iteratively refined
FR-10	Persist a profile across devices/accounts	Not implemented

2.4 Non-Functional Requirements

- Accuracy: exact claims should come from structured facts or strongly matching evidence.
- Responsiveness: routine conversational turns should bypass retrieval; indexed turns should remain usable on free or low-cost provider tiers.
- Maintainability: ingestion, retrieval, API orchestration, and UI should remain separable.
- Security: privileged keys remain server-side and are never included in client assets or documentation.
- Legal restraint: store small retrieval chunks and derived facts, not reconstructed full articles.
- Accessibility: preserve readable contrast, keyboard submission, focus behavior, scalable text, and mobile-safe touch targets.

3. Complete Feature Inventory

Implemented capabilities, their user value, technical shape, dependencies, and logical next step.

3.1 Persona-Styled Entrance Screen

Purpose: Creates a game-like first impression and transitions into the actual tool rather than a conventional marketing page.

User benefit: Users immediately understand the domain and tone.

Technical implementation: Shared HTML/CSS uses supplied visual assets, layered geometric panels, a SEES portrait seal, entrance transitions, and motion-reduction rules.

Dependencies: Static assets, CSS animations, browser storage.

Future improvement: Add a short skip setting and performance-aware animation quality.

3.2 Conversational Chat Interface

Purpose: Provides the primary experience for asking questions and reading multi-turn answers.

User benefit: The guide feels like an assistant instead of a document search.

Technical implementation: The active page injects index.html and loads public/script.js; messages render as user and assistant bubbles with a persistent composer.

Dependencies: Next.js page shell, JavaScript, CSS, /api/chat.

Future improvement: Migrate the active shell to a single typed React implementation after visual behavior stabilizes.

3.3 Casual Conversation Path

Purpose: Allows greetings, thanks, meta questions, and small talk to behave normally.

User benefit: Users can speak naturally without every turn becoming a database search.

Technical implementation: The API detects casual intent and returns a direct response before retrieval.

Dependencies: Intent heuristics and Groq controller.

Future improvement: Evaluate a compact intent model or structured classifier against real transcripts.

3.4 Intent and Action Controller

Purpose: Determines whether to answer directly, clarify, search guides, search facts, or search both.

User benefit: Vague language receives intelligent handling and exact questions receive evidence.

Technical implementation: A controller schema exposes five actions and domain intents, merging deterministic analysis with an LLM decision.

Dependencies: Groq-compatible chat API, route-level schemas.

Future improvement: Add controller evaluation fixtures and confidence calibration.

3.5 Clarification System

Purpose: Requests the smallest missing detail needed to make advice useful.

User benefit: Users are guided rather than rejected with 'provide more information'.

Technical implementation: Intent-specific follow-ups cover level, party, boss, month/date, and general blockers; prompts are limited and sanitized.

Dependencies: Controller plus profile extraction.

Future improvement: Rank clarification questions by expected information gain.

3.6 Session Context Memory

Purpose: Carries recent messages and inferred player details through the current session.

User benefit: Follow-up questions can refer to level, party, month, difficulty, playstyle, and recent bosses.

Technical implementation: The browser stores a player profile in sessionStorage and sends a rolling history window with each request.

Dependencies: Browser sessionStorage, ChatRequest history/profile fields.

Future improvement: Add optional user-controlled persistent profiles with deletion/export controls.

3.7 Player Profile Extraction

Purpose: Infers useful game state from ordinary language.

User benefit: The user does not need to complete a settings form before receiving personalized advice.

Technical implementation: The API merges explicit profile values with detected month, level, difficulty, party, playstyle, and recent boss updates.

Dependencies: Intent analysis and response companion metadata.

Future improvement: Add a profile review panel so users can correct mistaken inference.

3.8 Source-Backed RAG

Purpose: Grounds exact and strategic responses in the indexed guide corpus.

User benefit: Answers become more accurate and traceable than model-only generation.

Technical implementation: The route dynamically builds context from structured facts and vector/keyword chunks, then asks Groq to synthesize an answer.

Dependencies: Supabase, pgvector, Hugging Face, Groq.

Future improvement: Increase structured fact coverage and add retrieval evaluations.

3.9 Structured Fact Priority

Purpose: Separates exact game mechanics from broad prose.

User benefit: Weaknesses, dates, rewards, floors, and recipes are less likely to be hallucinated.

Technical implementation: Facts are typed, linked to entities and sources, ranked by confidence and source credibility, and searched before chunks.

Dependencies: entities/facts tables, searchFacts.ts.

Future improvement: Add conflict resolution and field-level freshness metadata.

3.10 Exact Weakness Safety Guard

Purpose: Prevents the model from inventing a weakness from loosely related text.

User benefit: A missing answer is safer than an incorrect tactical instruction.

Technical implementation: Exact affinity questions require structured affinity support; otherwise the route asks for a variant/floor or explains the evidence gap.

Dependencies: Controller, fact retrieval, subject-source matching.

Future improvement: Expand deterministic extraction and build an affinity regression suite.

3.11 Hybrid Chunk Retrieval

Purpose: Finds relevant guide prose even when wording differs.

User benefit: Broad strategy and walkthrough questions can be answered without exact keyword matching.

Technical implementation: The retriever combines pgvector similarity with keyword fallback, exact phrase boosts, deduplication, and source credibility tie-breaking.

Dependencies: match_chunks RPC, embedding service, Supabase.

Future improvement: Introduce reranking and query expansion while measuring latency.

3.12 Collapsible Sources

Purpose: Keeps references available without letting them dominate the response.

User benefit: Users can verify an answer when they care, while normal chat remains clean.

Technical implementation: Assistant cards render source metadata inside an expandable details control and keep retrieval terminology out of prose.

Dependencies: ChatResponse sources, static UI renderer.

Future improvement: Add cited claim highlighting and source freshness.

3.13 Suggested Replies

Purpose: Offers relevant next messages after clarifications or answers.

User benefit: Mobile users can continue quickly without typing long details.

Technical implementation: The API returns optional first-person suggestedPrompts; client code renders them as buttons and rejects assistant-style questions.

Dependencies: Companion response metadata.

Future improvement: Generate prompts from unresolved profile fields and current goal.

3.14 Responsive Mobile Composer

Purpose: Keeps the input usable around iOS browser chrome and the software keyboard.

User benefit: The primary action remains visible and readable on phones.

Technical implementation: CSS breakpoints, pointer detection, visualViewport listeners, keyboard-open state, dynamic viewport sizing, and larger mobile composer dimensions.

Dependencies: Modern browser Visual Viewport API.

Future improvement: Test across Safari/Chrome device matrix and add safe-area telemetry.

3.15 Static Preview and Mock Fallback

Purpose: Allows visual development without live credentials.

User benefit: The interface remains demonstrable and debuggable offline.

Technical implementation: The static shell calls a configured API URL or /api/chat and falls back to mock Persona-style data when unavailable; USE MOCK_CHAT can force mock mode.

Dependencies: index.html, script.js, environment variables.

Future improvement: Make mock mode visually explicit only in developer builds.

3.16 Balanced Source Discovery

Purpose: Expands the corpus beyond a short hand-maintained URL list.

User benefit: More questions can be answered from real guide pages.

Technical implementation: Curated seeds feed an allowlisted crawler that scores links, balances domains/categories, rejects irrelevant paths, and caps total pages.

Dependencies: Cheerio, sourceCatalog.ts, discoverSources.ts.

Future improvement: Add crawl manifests and incremental recrawl scheduling.

3.17 Respectful Fetching and Local Cache

Purpose: Reduces repeated requests and honors publisher rules.

User benefit: Ingestion is more reliable and less disruptive to source sites.

Technical implementation: robots.txt checks, rate limits, timeouts, an explicit user agent, raw-page cache, and an IGN reader fallback are implemented.

Dependencies: robots-parser, fetch, local filesystem.

Future improvement: Add retry budgets, conditional requests, and per-domain health metrics.

3.18 Content Cleaning and Table Preservation

Purpose: Turns guide HTML into compact evidence without navigation and advertising noise.

User benefit: Retrieval focuses on useful game information.

Technical implementation: Scripts, styles, menus, ads, comments, breadcrumbs, related content, and sharing UI are removed; meaningful paragraphs, headings, and tables are preserved.

Dependencies: Cheerio, extractContent.ts.

Future improvement: Add per-site extraction fixtures and DOM-drift alerts.

3.19 Chunking, Hashing, and Synchronization

Purpose: Creates retrieval units with stable identities.

User benefit: Updates do not continually duplicate the index.

Technical implementation: Sections become roughly 240-700 token chunks; SHA-256 hashes enforce uniqueness; safe sync deletes stale chunks only after replacement succeeds.

Dependencies: chunkText.ts, embedChunks.ts, database constraints.

Future improvement: Add semantic boundary scoring and chunk-version lineage.

3.20 Structured Fact Extraction

Purpose: Derives typed records from guide content.

User benefit: The chatbot can answer exact questions with a stronger evidence model.

Technical implementation: Zod-valid LLM extraction is combined with deterministic affinity-table parsing, evidence checks, confidence thresholds, and deduplication.

Dependencies: Groq/OpenAI-compatible model, Zod, Supabase.

Future improvement: Broaden deterministic parsers for Social Links, requests, items, and schedules.

3.21 Ingestion Audit and Backfill Tools

Purpose: Makes corpus health observable and recoverable.

User benefit: The developer can identify missing categories and rebuild facts without refetching every page.

Technical implementation: CLI scripts report coverage and support clear/rebuild fact backfills, dry runs, list mode, force refresh, sync, and page limits.

Dependencies: tsx scripts and Supabase.

Future improvement: Store audit snapshots and regression thresholds in CI.

3.22 Terminal Q&A Workflow

Purpose: Provides a fast backend-only test path.

User benefit: Retrieval and generation can be debugged independently from the visual UI.

Technical implementation: npm run ask invokes a simpler terminal pipeline using the same retrieval building blocks.

Dependencies: src/ask.ts, environment configuration.

Future improvement: Unify terminal orchestration with the production controller to avoid behavioral drift.

3.23 Feature Status Summary

Area	Implemented	Partial / experimental	Not yet implemented
Conversation	Multi-turn session history, intent, clarification, casual chat	Profile inference quality	Cross-device memory and accounts
Retrieval	Fact-first plus hybrid chunk search	Corpus breadth and fact density	Automated evaluation and reranking service
Interface	Entrance, chat, sources drawer, responsive composer	Static/React duality	Unified typed React UI and accessibility audit
Operations	Ingest, backfill, audit, static mock mode	Manual ingestion runs	Scheduled pipelines, dashboards, alerting

4. System Architecture

A layered view of the active application, retained prototype, services, and data boundaries.

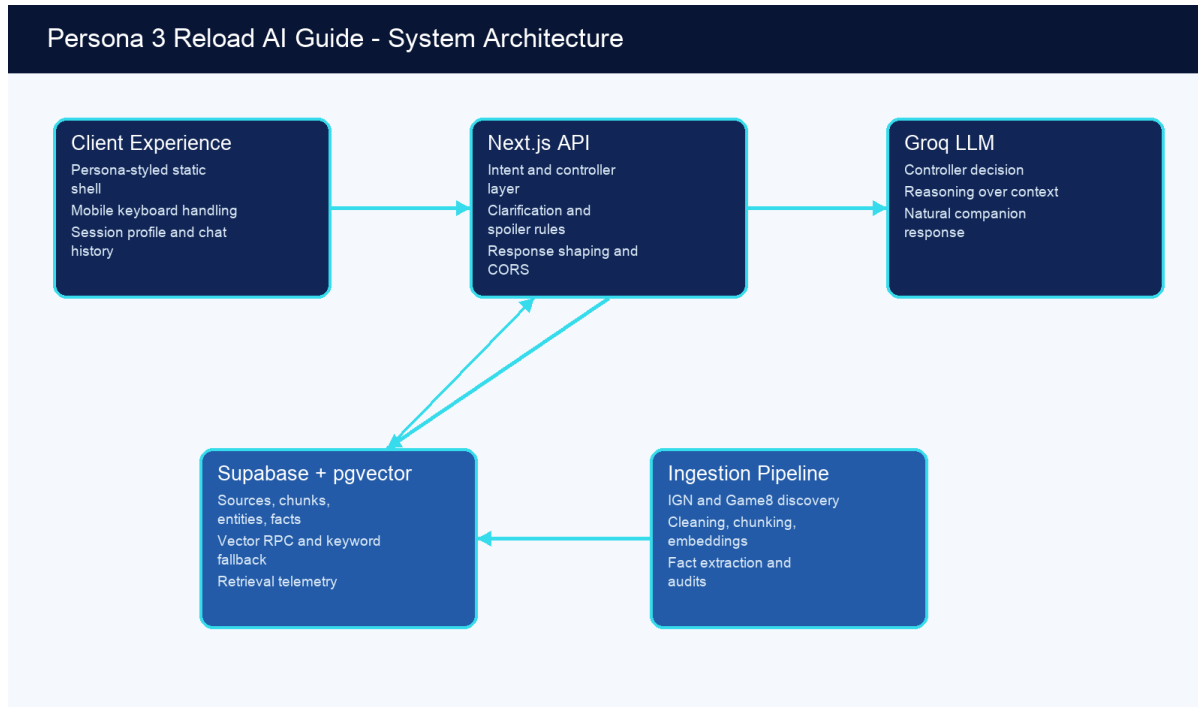


Figure 4-1. High-level architecture and primary data flow.

4.1 Frontend Architecture

The active rendering path is intentionally unusual. `app/page.tsx` reads the repository's `index.html`, extracts its body, injects that markup into the Next.js page, and loads `/script.js`. `app/layout.tsx` links `/styles.css`. This keeps the mature static visual implementation identical between a local static server and Vercel while still allowing the Next.js backend to share the deployment.

The repository also contains a componentized React prototype: `ChatShell`, `Sidebar`, `ChatInput`, `MessageBubble`, `GuideAnswerCard`, `SourceCard`, `WeaknessTable`, `PersonaBackground`, `LoadingPersona`, and `SuggestionButton`. These components demonstrate a cleaner future ownership boundary but are not the authoritative active shell today.

Frontend concern	Current implementation
Routing	Single primary page through Next.js App Router; POST API route under <code>/api/chat</code> .
State	Plain JavaScript state, <code>sessionStorage</code> profile, in-memory rolling chat history.
Styling	Large shared CSS system with desktop/mobile breakpoints, fixed Persona-inspired geometry, animations, and accessibility fallbacks.
Assets	Locally bundled JPEG menu art and a SEES portrait seal; no runtime third-party image dependency.

Frontend concern	Current implementation
Input behavior	Enter to submit, Shift+Enter for multiline behavior where supported, visualViewport keyboard adjustment.
Static compatibility	index.html + styles.css + script.js can run through a basic HTTP server and point to a remote API.

4.2 Backend Architecture

The backend is a serverless-friendly Next.js route. It validates requests, applies CORS, interprets conversation state, decides whether retrieval is necessary, builds context, invokes the chat provider, normalizes the model response, and returns a UI-oriented contract.

- Controller layer: intent, ambiguity, action, spoiler caution, profile updates, and suggested replies.
- Retrieval orchestration: fact search, chunk search, source deduplication, and retrieval logging.
- Generation layer: structured JSON response from an OpenAI-compatible endpoint, currently configured for Groq.
- Fallback layer: external RAG endpoint support, direct RAG, mock response, and explicit error/empty modes.
- Safety layer: exact-weakness support guard, response sanitization, confidence bounds, and no secret exposure.

4.3 Service and Deployment Topology

Service	Responsibility	Trust boundary
Vercel / Next.js	Hosts frontend and /api/chat server route	Public application; server environment holds secrets
Supabase	Postgres records, vector index, RPC search, retrieval logs	Service-role access is server-only
Hugging Face Inference	Produces 384-dimensional query and ingestion embeddings	API token remains server/ingestion-side
Groq API	Controller and final response generation	Receives user prompt, compact history, profile, and retrieved context
IGN and Game8	Authoritative external guide material	Fetches only during ingestion under allowlist/rate-limit rules
Local cache	Stores raw fetched pages during development	Gitignored; not served to users

4.4 Architectural Tension: Shared Static Shell vs. React

ENGINEERING DECISION

Keeping the static shell reduced deployment drift during rapid visual iteration, but it duplicates script/style files and weakens type safety. The practical migration path is to freeze visual acceptance tests, port one region at a time into ChatShell, and remove the injected HTML only after desktop and mobile parity is proven.

5. RAG System Documentation

The complete path from third-party guide discovery to a grounded conversational answer.

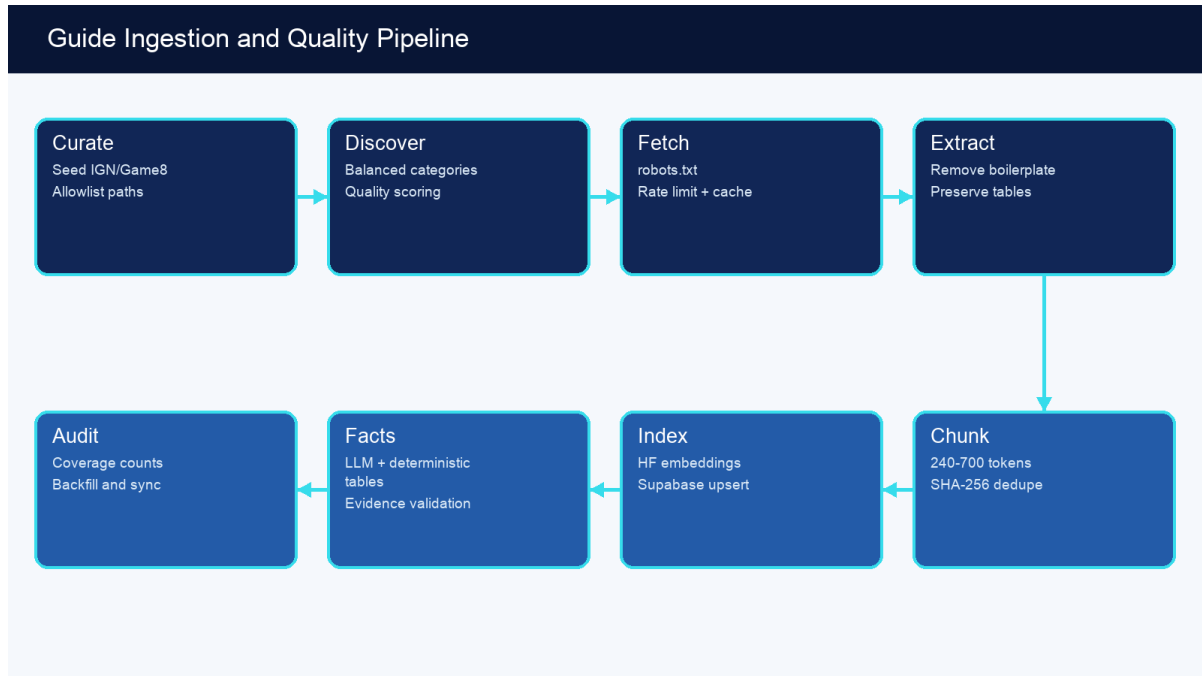


Figure 5-1. Offline ingestion and quality-control pipeline.

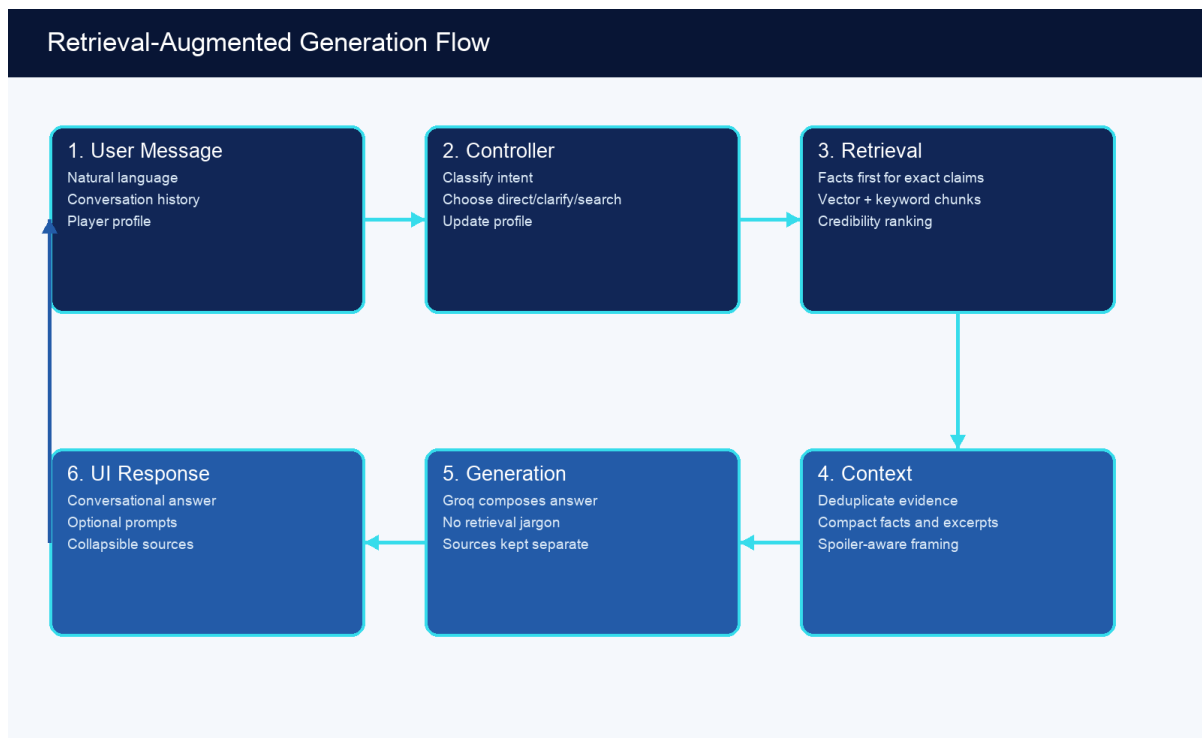


Figure 5-2. Online question routing, retrieval, reasoning, and presentation.

5.1 Data Collection

Curated seeds in `src/ingest/urls.ts` establish known high-value entry points. The discovery layer follows only approved IGN Persona 3 Reload wiki paths and Game8 Persona 3 Reload paths. It rejects generic news, comments, login pages, reviews, videos, unrelated games, and DLC-specific paths when those do not match the intended corpus.

Source metadata includes title, canonical URL, domain, category, source type, and credibility rank. IGN is assigned a stronger default rank than Game8 when otherwise comparable sources conflict. The design does not assume any publisher is infallible; credibility ranking is a deterministic tie-breaker, not proof.

5.2 Discovery and Fetching

1. Discover links from selected seed pages while maintaining domain and category balance.
2. Score high-value paths using terms associated with bosses, Tartarus, Personas, Social Links, requests, classroom answers, beginner strategy, and fusion.
3. Canonicalize URLs before comparison so tracking parameters and fragments do not produce duplicate sources.
4. Check `robots.txt` before fetching and use a descriptive ingestion user agent.
5. Rate-limit requests and cache raw responses under `.cache/raw-pages`.
6. For IGN, attempt the `r.jina.ai` reader representation first and fall back to direct HTML when required.

5.3 Content Processing

The extractor removes navigation, advertisements, scripts, styles, comments, breadcrumbs, newsletter prompts, related-content modules, video shells, and share controls. It keeps document title, meaningful headings, paragraphs, and table content. Game8 pages use a site-specific article root where available.

Tables receive special handling because affinity and schedule information is often lost when HTML is flattened. Cells are converted into explicit label/value text so downstream extraction can distinguish a boss's own affinity row from strategy prose about summoned enemies.

5.4 Chunking Strategy

Property	Current rule	Reason
Target size	Approximately 500 tokens	Enough context for strategy while remaining focused
Accepted range	Roughly 240-700 tokens; very small useful fragments may pass at about 70 tokens	Avoid both tiny context fragments and broad article-sized chunks
Boundary	Section headings and accumulated	Retains local semantic topic

Property	Current rule	Reason
	paragraphs	
Identity	SHA-256 of source URL, section title, and text	Stable deduplication and safe synchronization
Synchronization	Delete stale source chunks only after replacements embed successfully	Avoid an empty index after a partial failure

5.5 Embeddings

The default embedding model is sentence-transformers/all-MiniLM-L6-v2 through Hugging Face feature extraction. It produces 384-dimensional vectors, matching chunks.embedding_vector(384) and the match_chunks RPC signature. Query embeddings are normalized to a single vector shape, dimensions are checked, and provider calls use exponential retry behavior.

MODEL MIGRATION CONSTRAINT

Changing the embedding model is a schema migration, not only an environment-variable edit. EMBEDDING_DIMENSIONS, the chunks column, the RPC argument, and all stored vectors must agree; existing chunks must be re-embedded.

5.6 Vector and Keyword Storage

Supabase pgvector stores embeddings beside source-linked chunks. The migration creates an IVFFlat cosine index with 100 lists. The production retriever also performs a keyword fallback and merges the two result sets, which protects against semantic misses for exact names and unusual game terminology.

5.7 Structured Fact Extraction

Structured facts are a second retrieval plane. An LLM extraction pass emits Zod-validated entity/fact candidates, but deterministic affinity-table parsing can override ambiguous prose. Candidate facts require supporting evidence, a minimum confidence threshold, and valid entity/fact types before insertion.

This distinction is central to accuracy. Broad strategy prose is appropriate for 'how should I approach this fight?' but unsafe for 'what is this enemy weak to?'. Exact affinity questions are therefore blocked from a confident answer unless structured affinity evidence exists.

5.8 Retrieval

7. Normalize the user query and infer likely entity terms and fact types.
8. Search entities and structured facts for exact factual intents.

9. Embed the query and call `match_chunks` for semantic evidence.
10. Run keyword search as a fallback or complement.
11. Boost exact phrase matches, deduplicate chunks, and prefer credible sources on ties.
12. Assemble a compact context package and record matched IDs in `retrieval_logs`.

5.9 Context Construction

`buildContext` combines fact matches and chunk matches into a concise evidence package. Sources are deduplicated separately for UI display. The final prompt receives only the evidence needed for the turn rather than full pages, which controls latency, model context usage, and article-reconstruction risk.

5.10 Response Generation

The Groq model is instructed to act as a normal chat assistant first and a Persona veteran second. It integrates facts naturally, does not say 'according to the database', avoids retrieval jargon, keeps mobile readability in mind, and returns a machine-readable structure containing answer text, optional sections/tables, confidence, missing information, profile updates, follow-up questions, and suggested prompts.

5.11 Corpus Checkpoint and Current Limitation

JUNE 6, 2026 PROJECT CHECKPOINT

A documented ingestion run selected 180 balanced pages, prepared 526 clean chunks and 459 fact candidates, embedded 526 chunks, and removed 280 stale chunks. A subsequent audit reported 184 source rows, 544 chunks, 14 entities, and 34 structured facts after the initial fact pass. These counts are a time-specific checkpoint, not a permanently current production total.

The principal bottleneck is now fact density, not page discovery. Broad guide coverage can retrieve relevant articles, but exact answers remain dependent on entity-level structured extraction. The next quality milestone is a validated fact backfill across enemies, bosses, requests, Social Links, schedules, items, and fusion rules.

5.12 Evaluation Strategy

Evaluation layer	Recommended metric	Example test
Discovery	Category/domain coverage and dead-link rate	At least one high-quality page for every major guide category
Extraction	Useful-text precision and table preservation	Affinity table remains distinguishable from nearby strategy text
Fact extraction	Precision by fact type	Priestess nullifies Ice; do not assign summoned-shadow

Evaluation layer	Recommended metric	Example test
		weaknesses to Priestess
Retrieval	Recall@k and exact-subject precision	Dancing Hand query ranks the correct subject page above generic boss guides
Generation	Groundedness and actionability	Answer contains only supported exact claims and a practical next step
Conversation	Clarification usefulness and memory accuracy	Vague team question asks level/party once, then uses the answer

6. Knowledge Base Documentation

Information categories, data shape, relationships, and retrieval risks.

Category	Primary shape	Typical relationships	Retrieval challenge
Personas	Entity + facts/chunks	Fusion recipes, skills, unlocks, level and role	Names overlap; recipes vary with special conditions
Bosses	Boss entity + affinities + strategy chunks	Location, phases, null/repel/drain, recommended party	Nearby enemy prose can contaminate boss facts
Enemies / Shadows	Enemy entity + affinity facts	Weakness, resistance, location/floor	Variants may share similar names
Social Links	Entity + schedule/answer/prerequisite facts	Availability, dialogue choice, unlock path	Calendar and spoiler sensitivity
Requests	Request entity + deadline/reward/prerequisite	Completion steps and reward	Request numbers and names must be normalized
Calendar Events	Activity/location + schedule facts	Date, time slot, prerequisite	Date parsing and route-dependent availability
Tartarus	Floor/location entities + chunks	Block ranges, gatekeepers, rescue events, exploration tips	Floor terminology and procedural variation
Items	Item entity + item_effect/reward facts	Effect, source, use	Generic names and acquisition context
Equipment	Equipment entity + item_effect/chunks	Stats, effect, shop/drop source	Version or progression availability
Party Members	Party member entity + strategy chunks	Role, strengths, team synergy	Recommendations depend on level and unlocked roster
Fusion Information	Persona facts + guide chunks	Recipe, prerequisite, inherited skills	Exact recipes and special fusions require structured support
Story Content	Guide chunks with spoiler caution	Progress guidance and ending requirements	High spoiler risk; progress inference can be wrong
Gameplay Mechanics	Mechanic entity + tips/chunks	Theurgy, shifts, ailments, combat systems	Broad concepts can match many unrelated questions
Achievements	Activity/request/item entities + chunks	Unlock conditions and completion planning	Cross-system dependencies

6.1 Entity Normalization

Entities preserve the display name, normalized name, type, and aliases. The unique `normalized_name/type` pair prevents one textual subject from becoming multiple records while still allowing an enemy and an item to share a name if their types differ.

6.2 Source Relationships

Every chunk and fact links back to a source. This supports credibility ranking, source display, stale-content cleanup, and future claim-level provenance. Source deletion cascades into dependent chunks and facts, making source synchronization predictable.

6.3 Conflict Handling

Current conflict behavior is primarily ranking-based: lower `credibility_rank` sources are preferred, and fact confidence provides a secondary signal. A mature implementation should model conflicting values explicitly, compare publication or last-checked time, and avoid silently selecting a value when equally credible sources disagree.

7. User Experience Design

How the interface balances game identity, speed, readability, and chat conventions.

7.1 Landing and Entrance Experience

The first screen is an immersive Persona-inspired entrance, not a promotional landing page. It uses the supplied character art, cyan/navy palette, angled geometry, layered shards, and an Enter Records action. Motion creates presentation energy while prefers-reduced-motion provides a restrained alternative.

7.2 Chat Experience

The chat area follows familiar LLM conventions: the user's message appears separately, the assistant response occupies a larger readable surface, the composer remains visually stable, and new responses scroll into view. Game-specific styling is applied around this behavior rather than replacing it.

The default empty state includes a portrait seal, a clear question, a short description, and a small number of starter prompts. These prompts are onboarding aids rather than a second navigation system.

7.3 Navigation

A vertical Quick Menu provides domain entry points such as Shadows, Bosses, Personas, Social Links, Requests, and Tartarus. Category actions populate or submit relevant prompt language instead of navigating away from the conversation. On narrow screens the menu becomes an overlay controlled by a familiar hamburger button.

7.4 Mobile Experience

Problem	Current response	Remaining risk
iOS keyboard reduces viewport	visualViewport height/offset updates CSS state and scroll position	Safari browser chrome varies by device and launch context
Desktop geometry overflows	Mobile breakpoints resize hero, prompts, chat card, and composer	Long localized labels may still require dynamic sizing
Composer hidden behind browser UI	Safe-area spacing and keyboard-open adjustments	Standalone PWA and in-app browser need separate testing
Source cards overwhelm answer	Sources are collapsible	Large result sets need source caps and grouping
Touch targets too small	Buttons expand on mobile	Full accessibility audit is still pending

7.5 Error Handling and Empty States

- Mock mode: returns predictable sample answers when the live service is unavailable or deliberately disabled.
- Empty retrieval: says the exact information is not yet supported and asks for a useful narrowing detail.
- Provider failure: returns an error-mode response without exposing credentials or raw backend internals.
- Ambiguous message: asks a domain-specific question rather than a generic rejection.
- Status probe: `__status__` reports whether the route is configured for RAG or mock behavior.

7.6 Accessibility Considerations

The interface includes visible text labels, keyboard submission, reduced-motion support, high contrast, scalable mobile layouts, and source details that use native disclosure behavior. The remaining work is a formal keyboard/focus audit, semantic landmark review, screen-reader labeling, contrast measurement on image-backed regions, and automated checks with axe or Lighthouse.

7.7 Design Tradeoff

GAME IDENTITY VS. TOOL ERGONOMICS

Persona-style typography, angles, and oversized headings create the desired atmosphere, but they can crowd small screens. The governing rule should be: preserve the identity in backgrounds, accents, transitions, and section shapes; preserve conventional chat behavior in reading order, input placement, focus, and scrolling.

8. Database Documentation

Supabase Postgres schema, pgvector search, relationships, and example data.

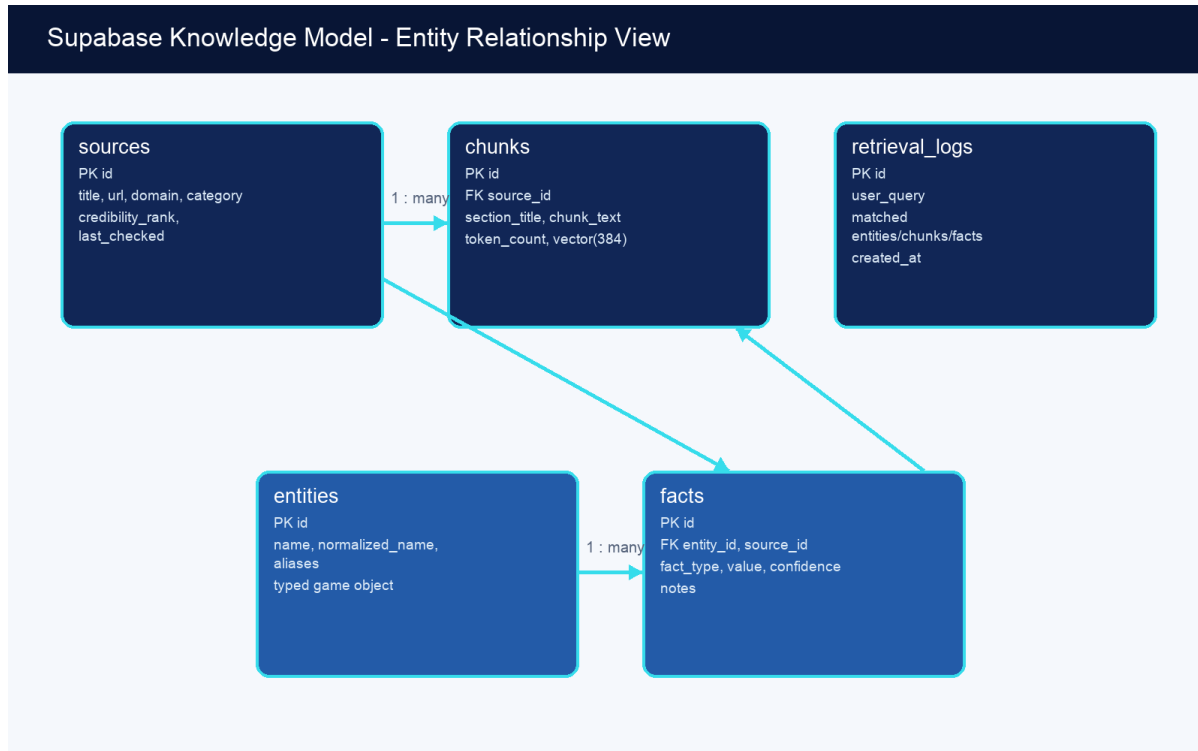


Figure 8-1. Knowledge-base entity relationship diagram.

8.1 sources

Registry of approved guide pages and their provenance.

Field	Type / constraint	Usage
id	uuid, primary key	Stable source identifier
title	text, required	Page title
url	text, unique, required	Canonical original URL
domain	text, required	Publisher domain
category	text, required	Guide taxonomy
source_type	text, default guide	Future source classification
credibility_rank	integer, default 50	Lower values receive preference
last_checked	timestamptz	Future freshness signal
created_at	timestamptz	Insertion time

Example record: {"title":"Boss Guides - Persona 3 Reload","url":"https://www.ign.com/wikis/persona-3-reload/Boss_Guides","domain":"ign.com","category":"bosses","credibility_rank":10}

8.2 chunks

Small, source-linked text units used for semantic and keyword retrieval.

Field	Type / constraint	Usage
id	uuid, primary key	Chunk identifier
source_id	uuid, FK -> sources	Cascade-owned source
section_title	text, optional	Local document heading
chunk_text	text, required	Clean retrieval content
chunk_hash	text, unique	SHA-256 deduplication identity
token_count	integer	Approximate chunk size
embedding	vector(384)	Semantic representation
created_at	timestampz	Insertion time

Example record: {"source_id":"...", "section_title":"Priestess Strategy", "chunk_text":"...", "token_count":486, "embedding":[384 values]}

8.3 entities

Normalized game subjects that structured facts describe.

Field	Type / constraint	Usage
id	uuid, primary key	Entity identifier
name	text, required	Display name
type	checked text	enemy, boss, persona, social_link, request, item, equipment, location, mechanic, party_member, tartarus_floor, activity, or skill
aliases	text[]	Alternative names
normalized_name	text, required	Search/dedupe identity
created_at	timestampz	Insertion time

Example record: {"name":"Priestess", "type":"boss", "aliases":["Arcana Priestess"], "normalized_name":"priestess"}

8.4 facts

Typed, source-supported claims for exact retrieval.

Field	Type / constraint	Usage
id	uuid, primary key	Fact identifier
entity_id	uuid, FK -> entities	Subject
source_id	uuid, FK -> sources	Provenance
fact_type	checked text	Weakness, resistance, strategy, schedule, recipe, reward, and other supported types
value	text, required	Normalized factual value

Field	Type / constraint	Usage
confidence	numeric 0..1	Extraction confidence
notes	text, optional	Qualification or evidence note
created_at	timestampz	Insertion time

Example record:

```
{"entity_id":"...", "source_id":"...", "fact_type":"nullifies", "value":"Ice", "confidence":0.96, "notes":"Explicit affinity table value"}
```

8.5 retrieval_logs

Developer-facing retrieval telemetry for understanding misses and matches.

Field	Type / constraint	Usage
id	uuid, primary key	Log identifier
user_query	text, required	Original question
matched_entities	jsonb	Entity match summary
matched_chunks	jsonb	Chunk match summary
matched_facts	jsonb	Fact match summary
created_at	timestampz	Request time

Example record: {"user_query":"What is Dancing Hand weak to?","matched_entities": [], "matched_chunks":[...], "matched_facts":[]}

8.6 Relationships and Constraints

- sources 1 -> many chunks; deleting a source removes its chunks.
- sources 1 -> many facts; deleting a source removes its derived facts.
- entities 1 -> many facts; deleting an entity removes its facts.
- entities are unique by normalized_name and type.
- facts are practically unique by entity, source, type, and a case-insensitive hash of value.
- chunks are globally unique by chunk_hash.

8.7 Vector RPC

match_chunks(query_embedding vector(384), match_count integer default 8, similarity_threshold float default 0.72) joins chunks to sources, filters by cosine similarity, orders first by source credibility and then vector distance, and returns chunk text plus source metadata.

8.8 Security Model

The application uses the Supabase service-role key only in server-side code and ingestion scripts. The initial migration does not encode a complete Row Level Security policy set, so browser access must never use the service role and public Data API exposure should be reviewed. For a public multi-user version, enable RLS, restrict direct table access, and keep retrieval behind the application API.

9. API Documentation

Request contracts, response contracts, behavior modes, and examples.

9.1 POST /api/chat

Purpose: Primary conversational endpoint. It may answer directly, clarify, retrieve from Supabase, call an external RAG endpoint, or return a controlled fallback.

Authentication: No end-user authentication is currently required. Provider and Supabase credentials are server-side environment variables.

CORS: Origins are controlled by ALLOWED_ORIGINS when configured; OPTIONS is supported.

Request Body

Field	Type	Required	Description
question	string	Yes	Current user message; __status__ is reserved for configuration probing
conversationId	string	No	Optional client conversation identifier
history	array of role/content	No	Recent user and assistant messages
playerProfile	object	No	Month, level, difficulty, active party, recent boss, Social Links, playstyle

Request Example

JSON

```
{
  "question": "My team feels weak",
  "conversationId": "session-123",
  "history": [
    { "role": "user", "content": "I am in June" },
    { "role": "assistant", "content": "I am in June" }
  ],
  "playerProfile": {
    "currentMonth": "June",
    "difficulty": "Normal",
    "activeParty": [
      "Yukari",
      "Junpei"
    ]
  }
}
```

Response Body

Field	Type	Description
answer	string	Primary natural-language response
sections	array	Optional titled supporting sections
tables	array	Optional exact lists, mainly weakness/item data
sources	array	Title, URL, and domain for expandable provenance
confidence	number	Bounded confidence signal
missingInfo	string	Information needed to improve the answer
retrievalMode	rag empty mock error	Operational result state
companion	object	Intent, profile updates, follow-up questions, suggested prompts

Field	Type	Description

Response Example

JSON

```
{
  "answer": "Who is on your active team, and what level are you? Once I know that, I can tell whether the problem is coverage, equipment, or being underleveled.",
  "sources": [],
  "retrievalMode": "rag",
  "companion": {
    "intent": "Team Building",
    "profileUpdates": {
      "currentMonth": "June"
    },
    "followUpQuestions": [
      "What level are you right now, and who is on your active team?"
    ],
    "suggestedPrompts": [
      "I am level 18 with Yukari and Junpei"
    ]
  }
}
```

9.2 OPTIONS /api/chat

Returns the route's CORS preflight response. It allows the static shell to call a separately hosted API when the origin is approved.

9.3 Status Probe

POSTing question='__status__' returns whether the route is configured for live RAG or mock mode. This is an internal UI capability, not a public health standard. A future deployment should expose a dedicated, rate-limited health endpoint with no configuration detail.

9.4 Response and Error Modes

Mode	Meaning	Client behavior
rag	Live controller/retrieval/generation completed	Render answer and optional sources
empty	Route connected but exact/relevant evidence was insufficient	Render helpful limitation and clarification
mock	Mock mode forced or live credentials unavailable	Render development sample answer
error	Unexpected provider, validation, or backend failure	Render safe retry guidance without raw secrets

9.5 External RAG Compatibility

If RAG_CHAT_ENDPOINT is configured and mock mode is disabled, the route can delegate to a separate service accepting POST {question, conversationId}. This provides an escape hatch for moving retrieval into another runtime, but it introduces a second contract that must remain versioned and tested.

10. AI System Design

Prompting, control flow, context injection, hallucination prevention, and companion behavior.

10.1 Two-Stage LLM Use

The application treats the LLM as both a controller and a writer. The controller decides whether the turn requires direct chat, clarification, guide retrieval, structured retrieval, or both. The final generator receives the selected evidence and produces the answer. This is more efficient and natural than sending every message through retrieval.

10.2 Intent Taxonomy

Intent	Typical examples	Preferred action
Enemy Weakness	What is Dancing Hand weak to?	Structured facts + exact guard
Boss Help	How do I beat Priestess?	Facts and guide chunks
Team Building	My team feels weak	Clarify profile, then guide reasoning
Fusion Advice	What should I fuse next?	Clarify level/goals; exact recipes require facts
Social Links	Who should I prioritize?	Profile/date clarification + guide search
Daily Schedule Planning	What should I do today?	Clarify month/date/free time
Tartarus Navigation	Where should I stop tonight?	Guide/floor facts
Quest Help	How do I finish request 29?	Structured request facts + guide
Story Guidance	How do I get the good ending?	Guide search with spoiler caution
Achievement Hunting	What am I missing for platinum?	Guide search and progress clarification
General Discussion	Hi / thanks / what can you do?	Answer directly

10.3 Prompt Engineering

- Role: knowledgeable Persona 3 Reload veteran and strategic coach.
- Behavior: normal conversational assistant first; retrieval is a hidden implementation detail.
- Accuracy: use structured facts for exact mechanics and do not invent unsupported values.
- Style: direct, practical, concise, mobile-readable, and free of generic filler.
- Clarification: ask one or two targeted questions when player state changes the recommendation.
- Spoilers: avoid later-game details unless requested or already established by player progress.
- Output: JSON fields are normalized before reaching the client.

10.4 Context Injection

The final generation prompt includes the current question, compact recent history, merged player profile, controller decision, structured facts, cleaned guide excerpts, and source metadata. It does not provide full articles. The model is told to treat retrieved material as evidence for exact claims and to keep provenance outside normal prose.

10.5 Hallucination Prevention

Control	What it prevents	Limit
Fact-first retrieval	Treating broad prose as exact data	Only works when facts have been extracted
Exact affinity guard	Invented weakness/resistance answers	May refuse a correct answer if the fact index is incomplete
Evidence validation	Unsupported extracted facts	Source HTML ambiguity can still pass
Low temperature	Unnecessary variation	Does not guarantee factuality
Credibility ranking	Lower-quality source winning a tie	Ranking is manually configured
Missing-info path	Confident answers from insufficient context	Must remain helpful rather than repetitive
Source separation	Retrieval jargon in prose	Citations do not prove every generated sentence

10.6 Response Formatting

Most turns should be one natural answer with short paragraphs. Sections are optional. Tables are reserved for exact affinity or item lists. Sources are rendered separately and collapsed. Suggested prompts must be phrased as user replies, not as assistant questions.

10.7 Memory Model

Memory is currently session-scoped and client-owned. The browser stores playerProfile in sessionStorage and keeps only a recent rolling message window. This is inexpensive and privacy-friendly, but it is not durable across devices, browsers, or cleared sessions. There is no user identity, long-term vector memory, or server-side conversation table.

10.8 Model Choice

The configured main chat model is llama-3.3-70b-versatile through Groq's OpenAI-compatible API. Structured fact extraction can use a faster model such as llama-3.1-8b-instant with delay and fallback controls. This split recognizes that ingestion is batch work while the live companion benefits from stronger reasoning and language quality.

11. Challenges and Engineering Decisions

Major problems, chosen solutions, tradeoffs, and the next engineering move.

Problem	Why difficult	Current solution	Tradeoff	Future improvement
Third-party page variability	IGN and Game8 use different layouts and change over time.	Allowlisted discovery, site-aware extraction, boilerplate removal, and table conversion.	Site-specific logic adds maintenance.	DOM fixtures and extraction drift alerts.
Retrieval accuracy	A semantically similar boss page may not describe the exact enemy.	Hybrid retrieval, exact phrase boost, entity facts, and exact-subject guards.	Some valid answers become unavailable until facts are backfilled.	Build gold query sets and reranking.
Affinity contamination	Strategy prose may mention another enemy's weakness near a boss name.	Explicit table values override nearby prose and evidence is checked.	Deterministic parsers are domain-specific.	Expand table parsers and regression tests.
Hallucinated exact answers	The LLM can sound confident even when evidence is weak.	Exact facts are required for weaknesses/dates/floors/recipes.	The assistant may ask for details more often.	Calibrated confidence and claim-level citations.
Vague user questions	The correct advice depends on level, party, date, and goal.	Intent-specific clarification plus profile memory.	Too many clarifications can feel obstructive.	Information-gain ranking and profile correction UI.
Context length and cost	Sending too much guide content increases latency and confusion.	Small chunks, source dedupe, compact context, retrieval only when needed.	Compression can omit nuance.	Adaptive context budgets and reranking.
Source disagreement	Two guides may differ or be updated at different times.	Credibility rank and confidence ordering.	Current model does not fully represent conflicts/freshness.	Versioned facts with last-checked and conflict records.
Mobile browser keyboard	iOS changes the visual viewport and browser chrome unpredictably.	visualViewport state, dynamic heights, safe-area spacing, mobile composer design.	Behavior varies by browser launch mode.	Device automation and PWA testing.
Static/React duplication	The polished shell and typed component prototype evolved separately.	Use the static shell as the active source of truth for now.	Duplicate CSS/JS and reduced type safety.	Incremental React migration with screenshot parity tests.
Free-tier provider limits	Batch fact extraction and live generation can hit rate limits.	Separate model choices, delays, retries, caching, and manual backfill sizes.	Ingestion takes longer and may require resumability.	Checkpointed job queue and provider abstraction.

11.1 Key Architectural Decision

REASON BEFORE RETRIEVE

The most important behavior change was replacing User -> Retrieve -> Summarize with User -> Intent/Context -> Decide -> Retrieve only if needed -> Reason -> Respond. This makes greetings and ambiguous situations feel conversational while preserving evidence for exact game knowledge.

12. Lessons Learned

What the project teaches about full-stack systems, RAG, user experience, and solo development.

12.1 Full-Stack Development

- A polished frontend can still fail if the API contract is unstable; response types and fallback modes should be designed early.
- Serverless deployment is convenient, but environment scope and redeployment behavior are part of application correctness.
- Keeping privileged data access behind one route is simpler and safer than exposing database clients to the browser.

12.2 AI Engineering

- A larger model improves language and reasoning but cannot compensate for missing or mislabeled evidence.
- Controller prompts and generator prompts solve different problems and should be evaluated separately.
- Structured output must be normalized and sanitized; provider-valid JSON is not automatically UI-valid behavior.

12.3 RAG Engineering

- More pages do not automatically produce more accurate answers. Fact density, entity normalization, and retrieval evaluation matter more after basic coverage.
- Hybrid retrieval is valuable because game names and identifiers are often better handled by lexical search than embeddings.
- Exact factual domains need typed records and deterministic parsing, not only vector chunks.

12.4 UI/UX

- Distinctive visual identity works best around familiar interaction patterns, not in place of them.
- Mobile software keyboards are a layout system of their own; testing screenshots without focus is insufficient.
- Sources should remain available but secondary to the answer.

12.5 Architecture

- Rapid prototypes can create duplicate active paths. Explicitly naming the source of truth prevents accidental drift.
- Batch ingestion deserves operational tooling from the beginning: dry runs, audit counts, sync safety, and resumability.
- A personal project still benefits from production boundaries when it touches secrets, external content, and public users.

12.6 Mistakes and What Would Be Done Differently

- Do not begin by treating all retrieved text as equally trustworthy. Introduce entity facts and evaluation fixtures earlier.
- Avoid maintaining two frontends longer than necessary. Establish screenshot baselines and migrate sooner.
- Do not expose operational labels such as RAG API or source-backed mode in the primary user experience.
- Build a test set before expanding the corpus. Otherwise ingestion success is measured by counts rather than answer quality.
- Keep secrets out of chats, screenshots, repositories, and client configuration from the first deployment step.

13. Future Feature Roadmap

A game-systems roadmap prioritized for user value, accuracy, and realistic solo-development effort.

#	Feature	Description	User value	Difficulty	Priority
1	Gold retrieval evaluation set	Curated questions and expected facts/sources	Very high	Medium	P0
2	Fact coverage dashboard	Coverage by entity/fact/category/source	Very high	Medium	P0
3	Enemy affinity backfill	Validated weaknesses/resistances for Shadows	Very high	High	P0
4	Boss mechanics backfill	Phases, affinities, party advice, locations	Very high	High	P0
5	Request database expansion	Deadlines, rewards, prerequisites, steps	High	Medium	P0
6	Profile correction panel	Let users review inferred level/party/date	High	Medium	P0
7	Claim-level citations	Map factual sentences to evidence	High	High	P1
8	Cross-encoder reranking	Rerank top chunks after hybrid retrieval	High	High	P1
9	Query rewrite service	Generate entity-aware alternate searches	High	Medium	P1
10	Persistent optional profile	Save progress across sessions/devices	High	High	P1
11	Fusion calculator	Exact normal/special fusion planning	Very high	High	P1
12	Fusion path optimizer	Cheapest or skill-preserving routes	High	High	P1
13	Persona compendium tracker	Track owned and missing Personas	High	Medium	P1
14	Social Link planner	Schedule priorities and prerequisites	Very high	High	P1
15	Daily schedule optimizer	Recommend activities by date and goals	Very high	Very high	P1
16	Spoiler level control	User-selectable strict/moderate/open detail	High	Medium	P1
17	Tartarus floor tracker	Remember highest floor and gate progress	High	Medium	P1
18	Party build recommendations	Skills, gear, roles, and coverage	High	High	P1
19	Equipment comparison	Compare owned gear and shop/drop options	Medium	High	P2
20	Elizabeth request tracker	Checklist with deadlines and rewards	High	Medium	P1
21	Achievement assistant	Progress checklist and missable warnings	High	High	P2
22	Calendar notification view	Upcoming full moons, deadlines, rescues	High	Medium	P2
23	Save-file import research	Parse supported save data locally	Very high	Very high	P3
24	Screenshot recognition	Read enemy/menu screenshots for context	High	Very high	P3
25	Voice companion	Hands-free questions during play	Medium	High	P3
26	PWA install mode	App-like mobile launch and offline shell	Medium	Medium	P2
27	Offline core facts	Bundle a compact weakness/request cache	High	High	P2
28	Conversation export	Download a play-session strategy log	Medium	Low	P2
29	Answer feedback	Helpful/incorrect controls tied to query logs	Very high	Medium	P1
30	Correction workflow	Developer review queue for bad facts	Very high	Medium	P1
31	Source freshness scheduler	Recheck changed pages incrementally	High	High	P2
32	Conflict detection	Flag contradictory facts across sources	High	High	P2
33	Additional trusted sources	Add fusion calculators and specialist references	High	Medium	P2
34	Model/provider fallback	Fail over between compatible providers	Medium	Medium	P2
35	Streaming responses	Display answer tokens progressively	Medium	Medium	P2
36	Accessible theme mode	Simplified typography, motion, and geometry	High	Medium	P2
37	Localization framework	Support translated UI and source sets	Medium	Very high	P3
38	Community corrections	Moderated evidence-linked submissions	Medium	Very high	P3
39	New Game Plus profile	Adjust advice for inherited progress	Medium	Medium	P3
40	DLC content packs	Explicit opt-in knowledge partitions	Medium	Medium	P3

13.1 Recommended Sequence

13. Accuracy foundation: evaluation set, fact dashboard, affinity/boss/request backfills, feedback loop.
14. Personalization foundation: profile correction, optional persistence, spoiler setting, progress trackers.
15. Game systems: fusion calculator, Social Link planner, schedule optimizer, party builds.
16. Operational maturity: scheduled freshness, conflict detection, provider fallback, streaming.
17. Experimental interfaces: screenshots, voice, save analysis, community contributions.

14. Scalability Considerations

How the architecture changes from one developer to a larger public audience.

Stage	Hosting and data	AI and retrieval	Operational changes
Personal use	Vercel hobby + Supabase free tier; manual ingestion	Groq/HF free or low-cost limits; small index	Local audits, no accounts, session memory
Small community	Paid limits as needed; scheduled ingestion	Request caching, provider rate control, evaluation suite	Basic analytics, abuse limits, feedback workflow
Hundreds of users	Connection pooling, regional strategy, larger vector resources	Streaming, reranking, prompt/result cache, async ingestion	Auth, quotas, monitoring, incident logs, RLS
Thousands of users	Dedicated database/vector capacity, CDN/PWA, queue workers	Model routing, batch embeddings, cost budgets, multi-provider failover	SLOs, tracing, data retention, privacy controls, deployment environments

14.1 Performance

- Skip retrieval for casual and purely conversational turns.
- Cache embeddings for repeated normalized queries.
- Cap facts/chunks before generation and rerank rather than sending more context.
- Stream final responses when the UI and provider contract are unified.
- Move ingestion into resumable background jobs with per-source checkpoints.

14.2 Cost Controls

- Use a strong live model only for turns that benefit from it; use deterministic logic for status and known templates.
- Use a lower-cost batch model for fact candidates, then validate deterministically.
- Cache source HTML, embeddings, controller decisions for safe repeated intents, and stable final facts.
- Record token/provider usage per request before public growth.
- Introduce daily provider budgets and graceful degradation rather than uncontrolled retries.

14.3 Security and Privacy at Scale

A larger audience changes the threat model. Add authenticated profiles only with explicit consent, RLS, deletion/export paths, retention limits, rate limiting, input size limits, prompt-injection defenses for retrieved text, and clear separation between public source content and private player state.

14.4 Vector Storage Evolution

The current Supabase pgvector design is appropriate for the corpus size. Growth should first tune indexes and query plans, then partition by game/content pack if necessary. A separate vector service is justified only when measured latency, filtering, scale, or operational needs exceed Postgres capabilities.

15. Portfolio Case Study

A polished, factual project narrative for resumes, GitHub, LinkedIn, and personal sites.

15.1 Project Summary

Built a full-stack AI companion for Persona 3 Reload that combines a distinctive game-inspired chat interface with a source-backed RAG pipeline. The system ingests approved IGN and Game8 guides, cleans and chunks content, generates Hugging Face embeddings, stores vectors and structured facts in Supabase, and uses a Groq-hosted LLM to decide when retrieval is needed and compose conversational strategy answers.

15.2 Technical Highlights

- Designed a two-stage controller/generator flow so normal conversation does not behave like search.
- Implemented hybrid retrieval across typed facts, pgvector similarity, keyword fallback, exact phrase boosts, and source credibility.
- Built a legal-safer ingestion pipeline with allowlists, robots checks, rate limiting, local caching, content cleaning, stable hashes, and safe synchronization.
- Added deterministic affinity table parsing and an exact-weakness guard to reduce tactical hallucinations.
- Created a responsive Persona-inspired interface with session context, collapsible citations, suggested replies, and iOS visual viewport handling.
- Deployed a server-side API boundary that protects Supabase service-role, embedding, and chat credentials.

15.3 Technology Stack

TypeScript, Next.js 16, React 19, Supabase Postgres, pgvector, Hugging Face Inference, Groq OpenAI-compatible chat completions, Cheerio, Zod, robots-parser, Tailwind/PostCSS, Framer Motion, Lucide, Vercel, and GitHub.

15.4 Resume Bullets

- Engineered a Persona 3 Reload conversational guide using Next.js, TypeScript, Groq, Hugging Face embeddings, Supabase, and pgvector, with hybrid fact/vector retrieval and source-linked responses.

- Built a respectful web-ingestion pipeline for 180+ curated/discovered guide pages, including robots checks, rate limiting, HTML cleaning, chunk hashing, vector synchronization, structured fact extraction, and corpus auditing.
- Reduced exact-answer hallucination risk through typed entity facts, deterministic affinity-table parsing, source credibility ranking, and an exact-weakness evidence guard.
- Designed a responsive game-inspired chat experience with session memory, intelligent clarification, optional citations, and mobile software-keyboard handling.

15.5 GitHub README Case Study Block

SUGGESTED COPY

Tartarus Guide is a source-backed Persona 3 Reload AI companion. Instead of searching long guide articles, players can describe what they are trying to do and receive concise, contextual strategy. The project combines a Next.js chat application with a Supabase/pgvector knowledge base, Hugging Face embeddings, Groq generation, and a custom ingestion pipeline for trusted IGN and Game8 material. It is a solo learning and portfolio project focused on conversational UX, retrieval accuracy, and game-domain reasoning.

15.6 LinkedIn / Personal Website Summary

I built a conversational Persona 3 Reload guide to explore what it takes to make RAG feel like an actual expert companion instead of a search box. The interesting work was not just connecting an LLM: it was cleaning guide data, separating exact facts from broad prose, designing retrieval safeguards, preserving player context, and making the experience usable on mobile while retaining the game's visual personality.

15.7 Honest Project Weaknesses

The corpus is broader than the structured fact layer, persistent profiles are not implemented, source conflicts are not modeled explicitly, automated retrieval evaluations are still needed, and the active frontend uses a static-shell bridge rather than the cleanest React architecture. Presenting these limits is part of the case study because they identify the real next engineering work.

16. Developer Retrospective

A personal engineering reflection rather than a product pitch.

16.1 Why This Was Built

The project began with a simple frustration: useful Persona 3 Reload information exists, but finding the right fragment while playing interrupts the experience. Building a companion created an opportunity to solve that small problem while learning the systems behind modern AI applications.

16.2 Most Interesting Technical Challenge

The hardest problem was not generating fluent text. It was deciding when fluent text was allowed to contain an exact claim. A related guide page can make an answer sound plausible while still being wrong. Introducing structured facts, deterministic affinity parsing, exact-subject checks, and clarification behavior changed the architecture from 'search and summarize' into a more defensible reasoning system.

16.3 Favorite Features

- The controller that lets a simple greeting remain a simple greeting.
- The source drawer, because it preserves trust without making the interface feel like a database viewer.
- The Persona-inspired entrance and portrait seal, because they make the tool feel made for this specific game.
- Safe chunk synchronization and deterministic table parsing, because they solve quiet backend problems that directly improve answer quality.

16.4 What Comes Next

The next phase should prioritize evidence quality over more visual features: establish a gold evaluation set, increase structured fact coverage, build a correction loop, and let users review the profile the assistant has inferred. Once those foundations are reliable, fusion and schedule planning become valuable higher-level systems rather than new surfaces built on uncertain data.

16.5 Advice for Similar Projects

- Choose a domain you genuinely care about; the edge cases will require patience.
- Do not measure a RAG system only by how many documents it indexed.
- Create typed representations for facts that must never be guessed.

- Design the missing-information response as carefully as the successful answer.
- Keep provider secrets and service-role credentials server-side from day one.
- Build evaluation examples before tuning prompts or changing models.
- Let the product identity be expressive, but keep core interaction patterns familiar.

CLOSING REFLECTION

This project is strongest when treated as an evolving engineering notebook: every incorrect answer becomes a retrieval test, every mobile layout failure becomes a UX constraint, and every new feature is an opportunity to make the companion more truthful, useful, and natural.

Appendix A. Developer Runbook

Common local, ingestion, validation, and deployment operations.

A.1 Local Application

Task	Command / location
Install dependencies	npm install
Run Next.js locally	npm run dev
Build production bundle	npm run build
Run production server	npm run start
Type-check	npm run typecheck
Static preview	python3 -m http.server 4173
Terminal question	npm run ask -- "What is Dancing Hand weak to?"

A.2 Ingestion

Task	Command
Normal ingestion	npm run ingest
Balanced broad sync	npm run ingest -- --max-pages=180 --skip-facts --sync
Dry run	npm run ingest -- --max-pages=180 --dry-run
Refresh raw cache	npm run ingest -- --force
Curated seeds only	npm run ingest -- --no-discover
Audit coverage	npm run ingest:audit
Backfill facts	npm run ingest:facts -- --clear-existing --max-fact-chunks=180

A.3 Deployment Checklist

- Confirm secrets exist in Production and Preview environment scopes as intended.
- Set USE MOCK_CHAT=false once live Supabase, embedding, and chat credentials are present.
- Run npm run typecheck and npm run build.
- Deploy the latest Git commit to Vercel without reusing stale build cache after environment changes.
- POST __status__ and confirm retrievalMode=rag.
- Run a greeting, vague clarification, exact affinity, boss strategy, and mobile keyboard smoke test.
- Verify no service role, Hugging Face, Groq, or GitHub token appears in client assets or logs.

Appendix B. Environment Variable Reference

Names and purposes only. Secret values must never be committed or embedded in client code.

Variable	Purpose	Scope
SUPABASE_URL	Supabase project base URL	Server/ingestion
SUPABASE_SERVICE_ROLE_KEY	Privileged server-side database key	Secret
EMBEDDING_PROVIDER	huggingface or compatible provider selector	Server/ingestion
EMBEDDING_API_KEY	Embedding provider credential	Secret
EMBEDDING_BASE_URL	Embedding endpoint base	Server/ingestion
EMBEDDING_MODEL	Embedding model identifier	Server/ingestion
EMBEDDING_DIMENSIONS	Expected vector length; currently 384	Schema-sensitive
CHAT_API_KEY	Groq/OpenAI-compatible chat credential	Secret
CHAT_BASE_URL	Chat completions API base	Server
CHAT_MODEL	Live companion model; configured for llama-3.3-70b-versatile	Server
FACT_EXTRACTION_MODEL	Batch fact extraction model	Ingestion
FACT_EXTRACTION_DELAY_MS	Rate-control delay for fact calls	Ingestion
INGEST_USER_AGENT	Descriptive crawler identity/contact	Ingestion
INGEST_DELAY_MS	Per-request crawl delay	Ingestion
RAW_CACHE_DIR	Local raw-page cache path	Ingestion
RAG_CHAT_ENDPOINT	Optional external RAG service	Server
USE MOCK_CHAT	Force mock behavior when true	Server
ALLOWED_ORIGINS	Permitted cross-origin frontend origins	Server

CREDENTIAL HYGIENE

Any credential pasted into a chat, screenshot, issue, or commit should be revoked and regenerated. This document intentionally contains no secret values.

Appendix C. Repository Map

The most important implementation files and their responsibilities.

Path	Responsibility
app/page.tsx	Active Next page; injects the static shell
app/api/chat/route.ts	Conversation controller, RAG orchestration, generation, fallback
index.html / public copy	Shared chat and entrance markup
script.js / public copy	Client state, rendering, mobile viewport, API integration
styles.css / public copy	Persona visual system and responsive layout
components/*	Retained typed React component prototype
src/db/client.ts	Supabase, embedding, and chat provider clients
src/retrieval/buildContext.ts	Fact/chunk assembly and retrieval logging
src/retrieval/searchFacts.ts	Entity/fact lookup
src/retrieval/searchChunks.ts	Vector and keyword hybrid search
src/ingest/discoverSources.ts	Balanced allowlisted link discovery
src/ingest/fetchPages.ts	robots, rate limiting, cache, fetch fallback
src/ingest/extractContent.ts	Readable content and table extraction
src/ingest/chunkText.ts	Token-sized chunks and stable hashes
src/ingest/embedChunks.ts	Embeddings, upsert, safe sync
src/ingest/extractFacts.ts	LLM and deterministic structured fact extraction
src/ingest/auditIndex.ts	Coverage auditing
src/ingest/backfillFacts.ts	Fact rebuild from existing chunks
supabase/migrations/001_init.sql	Database schema and vector RPC

Appendix D. Testing Strategy

A practical test pyramid for the next stage of the project.

D.1 Unit Tests

- URL allowlist, canonicalization, categorization, and rejection rules.
- Chunk boundaries, token estimates, hash stability, and synchronization decisions.
- Intent classification and player-profile extraction fixtures.
- Suggested-prompt sanitation and response normalization.
- Exact-affinity support checks and subject matching.

D.2 Integration Tests

- Supabase migration and match_chunks behavior against a disposable project or local Postgres.
- Embedding dimension and provider response-shape validation.
- Ingestion fixture from saved HTML through chunks and facts.
- POST /api/chat in direct, clarify, RAG, empty, mock, and provider-error modes.

D.3 End-to-End Tests

- Desktop entrance, chat submission, answer rendering, source expansion, and scrolling.
- Mobile menu, prompt cards, composer focus, keyboard open/close, submission, and auto-scroll.
- Cross-origin static frontend calling the Vercel API.
- Regression screenshots at representative desktop and mobile viewports.

D.4 Accuracy Regression Set

Maintain a versioned JSON dataset with question, intent, profile, expected clarification or answer facts, forbidden claims, and expected source subjects. Run it before changing prompts, models, chunking, fact extraction, or retrieval thresholds.

Appendix E. Glossary

Project-specific terminology.

Term	Definition
Chunk	A small source-linked section of cleaned guide text used for retrieval.
Embedding	A numeric vector representing semantic meaning; currently 384 dimensions.
Entity	A normalized game subject such as a boss, Persona, request, or item.
Fact	A typed, source-supported claim attached to an entity.
Hybrid retrieval	Combining semantic vector search with lexical/keyword search.
RAG	Retrieval-Augmented Generation: retrieving evidence before generating an answer.
Controller	The decision stage that chooses direct response, clarification, or retrieval.
Groundedness	The degree to which answer claims are supported by provided evidence.
Credibility rank	A manually assigned ordering signal; lower values receive preference.
Exact affinity guard	A rule that blocks exact weakness/resistance answers without structured support.
Session profile	Browser-session player context such as month, level, party, and difficulty.
Static shell	The HTML/CSS/JavaScript UI shared by local preview and the active Next page.